



Hashing in C# met BCrypt.NET

[Luisteren](#)

Gepubliceerd: vrijdag 12 december 2025 door [Lutt, Tim](#)

Stap 0 – Waarom wachtwoorden hashen?

Heel kort:

- **Nooit** wachtwoorden als gewone tekst in de database opslaan.
- Met **BCrypt** wordt het wachtwoord:
 - gehasht (een eenrichtingsfunctie),
 - mét een unieke salt,
 - en met een instelbare “work factor” (traag genoeg om brute force moeilijk te maken).

Je slaat alleen de **hash** op, niet het originele wachtwoord.

Stap 1 – BCrypt.NET-Next installeren via NuGet

Je hebt het pakket **BCrypt.NET-Next** nodig, gemaakt door o.a. Chris McKee, Ryan D. Emerl en Damien Miller.

1. Rechtsklik op je **project** in Solution Explorer.
2. Kies “**Manage NuGet Packages...**”.
3. Ga naar het tabblad **Browse**.
4. Zoek naar: `BCrypt.Net-Next`
5. Selecteer het pakket **BCrypt.Net-Next**.
6. Klik op **Install**.

Daarna kun je in je C# code `BCrypt.Net.BCrypt` gebruiken. De uitleg die hieronder volgt is voor de versie 4.x van deze package. Er is momenteel een nieuwe versie in ontwikkeling, daarvan kun je de README en broncode vinden op [GitHub](#).

Stap 2 – User-model meteen plek voor de hash

Je hebt een User-klasse nodig met in ieder geval:

- Username (string)
- PasswordHash (string) – of gewoon Password, zolang je maar **de hash** opslaat.

Bijvoorbeeld:

csharp

```
public class User
{
    public int Id { get; set; }

    public string Username { get; set; }

    // Dit is NIET het plain-text wachtwoord, maar de hash:
    public string Password { get; set; } // of: public string PasswordHash { get; set; }

    public string Role { get; set; }
}
```

Belangrijk: Noem het gerust PasswordHash zodat voor iedereen duidelijk is dat het om een hash gaat.

Stap 3 – Bij registratie: wachtwoord hashen en opslaan

Stel dat je bij registratie de ingevulde waarden hebt:

- enteredUsername
- enteredPassword
- selectedRole

Dan maak je een nieuwe gebruiker aan en hash je het wachtwoord met:

```
csharp
```

```
var newUser = new User
{
    Username = enteredUsername,
    Password = BCrypt.Net.BCrypt.HashPassword(enteredPassword),
    Role = selectedRole
};
```

Daarna sla je newUser op in je database, bijvoorbeeld met Entity Framework:

```
csharp
```

```
db.Users.Add(newUser);
db.SaveChanges();
```

Wat er gebeurt:

- HashPassword genereert automatisch een **salt**.
- Maakt een hash van het wachtwoord + salt.
- Resultaat is een **lange string** die alles bevat wat nodig is om later te verifiëren.

Je slaat alleen die string op.

Stap 4 – Bij inloggen: gebruiker zoeken op username

Bij het inloggen wil je het volgende doen:

1. Zoek de gebruiker **alléén** op gebruikersnaam (niet op wachtwoord).
2. Gebruik vervolgens **BCrypt.Verify** om het ingevoerde wachtwoord te vergelijken met de opgeslagen hash.

Voorbeeld:

csharp

```
// When we've hashed the password, we want to only select the User by their username
var user = db.Users.FirstOrDefault(u =>
    u.Username.ToLower() == enteredUsername.ToLower());
```

- enteredUsername is wat de gebruiker intypt.
- Als user null is, bestaat die username niet.

Stap 5 – Bij inloggen: wachtwoord verifiëren met BCrypt.Verify

Nu controleer je het wachtwoord:

csharp

```
if (user == null || !BCrypt.Net.BCrypt.Verify(enteredPassword, user.Password))
{
    ShowError("Incorrect username or password!");
    return;
}
```

Wat BCrypt.Verify doet:

- Neemt het **ingevoerde** wachtwoord (enteredPassword),
- Haalt de salt en instellingen uit de opgeslagen hash (user.Password),
- Rekent een nieuwe hash uit,
- Vergelijkt die met de opgeslagen hash.
- Geeft **true** als het klopt, anders **false**.

Zo hoef jij geen salt of iets anders zelf te beheren — dat regelt BCrypt voor je.

Stap 6 – Wat te doen na succesvolle login?

Als de verificatie slaagt (dus niet in de bovenstaande if terecht komt) kun je de gebruiker inloggen:

- **Bij een API bijvoorbeeld een Session Token genereren en opslaan bij de gebruiker.** Je stuurt de Session Token naar de gebruiker, zodat die bij ieder volgend verzoek meegestuurd kan worden om te bewijzen dat dit de ingelogde gebruiker is.
- Of in een volledige lokale WinUI applicatie (zonder API) de user in een statische eigenschap opslaan (bijv.: User.LoggedInUser), zodat duidelijk is dat de gebruiker ingelogd is.